

API Documentation — TravelPort

Base URL (local): `http://localhost:5000/api/v1`

Base URL (Docker): `http://localhost/api/v1`

Swagger UI (local): `http://localhost:5000/swagger`

Swagger UI (Docker): `http://localhost/api/swagger`

Auth: Authorization: Bearer <jwt_token>

Authentication

Method	Endpoint	Auth	Description
POST	<code>/auth/register</code>	None	User registration
POST	<code>/auth/login</code>	None	Email/password login
POST	<code>/auth/refresh</code>	None	Refresh access token
POST	<code>/auth/logout</code>	Required	Invalidate tokens
POST	<code>/auth/forgot-password</code>	None	Send reset email
POST	<code>/auth/reset-password</code>	None	Reset with token
POST	<code>/auth/verify-email</code>	None	Verify email OTP

POST /auth/register

```
Request:
{ "name": "Sample User", "email": "user@example.invalid", "phone": "9000000000", "password": "
replace-with-strong-password" }

Response 201:
{ "success": true, "message": "Registration successful. Please verify your email." }
```

POST /auth/login

```
Request:
{ "email": "user@example.invalid", "password": "replace-with-strong-password" }

Response 200:
{
  "success": true,
  "data": {
    "accessToken": "eyJ...",
    "refreshToken": "abc...",
    "expiresIn": 900,
    "user": { "userId": "uuid", "name": "Sample User", "email": "user@example.invalid", "role": "User" }
  }
}
```

POST /auth/forgot-password

```
Request:
{ "email": "user@example.invalid" }
```

```
Response 200:
{
  "success": true,
  "message": "If an account with that email exists, a password reset link has been sent."
}
```

The reset link points to `http://localhost:5173/reset-password?token=...` and expires in **1 hour**.

When SMTP email is disabled (`Email__Enabled=false`), the backend logs the reset link instead of sending email.

POST /auth/reset-password

```
Request:
{ "token": "raw-reset-token", "newPassword": "replace-with-new-strong-password" }

Response 200:
{
  "success": true,
  "message": "Password reset successful."
}
```

On success, all existing refresh tokens for that user are revoked.

Flights

Data source: 900+ DB seed flights generated programmatically across 42 bidirectional routes with 7 airlines: IndiGo (6E), SpiceJet (SG), Air India (AI), Vistara (UK), Akasa Air (QP), Air India Express (IX), Go First (G8). Duffel live results can optionally be merged when `Duffel.Enabled=true`.

Method	Endpoint	Auth	Description
GET	/flights/search	None	Search available flights
GET	/flights/:id	None	Flight details
POST	/flights/book	Required	Book a flight
GET	/flights/popular	None	Popular routes

GET /flights/search

```
Query Params:
origin=BOM&destination=DEL&departureDate=2025-12-01&passengers=1&cabinClass=Economy
&maxPrice=10000&maxStops=1&airlines=6E,AI&sortBy=price&page=1&pageSize=20

Response 200:
{
  "success": true,
  "data": {
    "flights": [
      {
        "flightId": "uuid",
        "airline": "IndiGo",
        "flightNumber": "6E-123",
        "source": "BOM", "destination": "DEL",
        "departureTime": "2024-12-01T06:00:00Z",
        "arrivalTime": "2024-12-01T08:15:00Z",
        "duration": 135,
        "stops": 0,
        "availableSeats": 45,
        "price": 4599.00,
        "class": "Economy"
      }
    ]
  }
}
```

```
}
],
"total": 24,
"page": 1,
"limit": 10
}
}
```

POST /flights/book

```
Request:
{
  "flightId": "uuid",
  "class": "Economy",
  "passengers": [
    { "name": "John Doe", "age": 30, "gender": "Male", "passportNo": "A1234567" }
  ],
  "contactEmail": "user@example.invalid",
  "couponCode": "SAVE100"
}
```

```
Response 201:
{
  "success": true,
  "data": {
    "bookingId": "uuid",
    "bookingRef": "TP2024001234",
    "status": "Pending",
    "totalAmount": 4599.00,
    "discountAmount": 100.00,
    "finalAmount": 4499.00,
    "paymentUrl": "/api/v1/payments/initiate/uuid"
  }
}
```

Hotels

Method	Endpoint	Auth	Description
GET	/hotels/search	None	Search hotels
GET	/hotels/:id	None	Hotel details (includes reviews array)
GET	/hotels/:id/rooms	None	Available rooms
POST	/hotels/:id/reviews	Required	Submit a hotel review (requires completed stay)
POST	/hotels/book	Required	Book a hotel

GET /hotels/search

```
Query Params:
city=Mumbai&checkin=2024-12-01&checkout=2024-12-03&guests=2&rooms=1
&minPrice=500&maxPrice=5000&rating=4&sort=price&page=1&limit=10
```

GET /hotels/:id

Returns full hotel details including a reviews array ordered by most recent.

```
Response 200:
{
  "success": true,
  "data": {
```

```

{id": "uuid",
"name": "The Grand Palace",
"city": "Mumbai",
"reviewScore": 4.3,
"reviewCount": 5,
"rooms": [...],
"reviews": [
{
{id": "uuid",
"userId": "uuid",
"userName": "John Doe",
"rating": 5,
"comment": "Excellent stay, very clean rooms.",
"createdAt": "2026-05-10T10:00:00Z"
}
]
}
}

```

POST /hotels/:id/reviews

Authenticated user must have a **completed** hotel stay (CheckOut in the past, status Confirmed) at this hotel. One review per user per hotel.

```

Request:
{ "rating": 5, "comment": "Excellent stay, very clean rooms." }

Response 201:
{
"success": true,
"message": "Review submitted successfully.",
"data": {
{id": "uuid",
"userId": "uuid",
"userName": "John Doe",
"rating": 5,
"comment": "Excellent stay, very clean rooms.",
"createdAt": "2026-05-19T10:00:00Z"
}
}
}

```

Validation rules:

- rating must be 1–5 (integer)
- comment must be non-empty
- User must have a confirmed, checked-out booking for this hotel
- User can only submit one review per hotel (returns 422 if duplicate)

Buses

Method	Endpoint	Auth	Description
GET	/buses/search	None	Search bus routes
POST	/buses/book	Required	Book a bus seat
POST	/buses/:busId/seats/lock	Required	Lock seats during selection (10 min)
DELETE	/buses/:busId/seats/lock	Required	Release previously locked seats
GET	/buses/:busId/seats/locked	Required	Get seats locked by other users

GET /buses/search

Query Params:
origin=Mumbai&destination=Pune&travelDate=2025-12-01&seats=1&pageSize=30

Response 200:

```
{
  "success": true,
  "data": [
    {
      "id": "uuid",
      "operator": "RedBus Express",
      "busType": "Volvo A/C Sleeper",
      "origin": "Mumbai", "destination": "Pune",
      "departureTime": "2025-12-01T22:00:00Z",
      "arrivalTime": "2025-12-02T04:00:00Z",
      "durationMinutes": 360,
      "availableSeats": 24,
      "price": 899.00,
      "acAvailable": true,
      "isRefundable": true,
      "amenities": "WiFi, Charging, Blanket",
      "rating": 4.3
    }
  ],
  "meta": { "page": 1, "pageSize": 30, "total": 12 }
}
```

Trains

Method	Endpoint	Auth	Description
GET	/trains/search	None	Search train routes
POST	/trains/book	Required	Book a train seat

GET /trains/search

Query Params:
origin=Mumbai&destination=Delhi&travelDate=2025-12-01&class=3A&passengers=1&pageSize=30

Response 200:

```
{
  "success": true,
  "data": [
    {
      "id": "uuid",
      "trainNumber": "12951",
      "trainName": "Mumbai Rajdhani Express",
      "origin": "Mumbai", "destination": "Delhi",
      "departureTime": "2025-12-01T17:00:00Z",
      "arrivalTime": "2025-12-02T09:55:00Z",
      "durationMinutes": 955,
      "classes": {
        "SL": { "className": "SL", "availableSeats": 120, "price": 715, "availability": "AVAILABLE" },
        "3A": { "className": "3A", "availableSeats": 64, "price": 1865, "availability": "AVAILABLE" },
        "2A": { "className": "2A", "availableSeats": 32, "price": 2750, "availability": "WL-3" },
        "1A": { "className": "1A", "availableSeats": 8, "price": 4625, "availability": "REGRET" },
        "CC": { "className": "CC", "availableSeats": 0, "price": 1595, "availability": "REGRET" }
      },
      "runningDays": "Daily",
      "availableSeats": 184,
      "isTatkal": false
    }
  ],
  "meta": { "page": 1, "pageSize": 30, "total": 8 }
}
```

Availability values: AVAILABLE · WL-N (waitlist) · RAC-N (reservation against cancellation) · REGRET (no seats)

Cabs

Method	Endpoint	Auth	Description
GET	/cabs/search	None	Search available cabs
POST	/cabs/book	Required	Book a cab

GET /cabs/search

Query Params:
origin=Mumbai&destination=Pune&pickupDateTime=2025-12-01T10:00:00&tripType=OneWay&pageSize=20

Response 200:

```
{
  "success": true,
  "data": [
    {
      "id": "uuid",
      "provider": "Ola",
      "cabType": "Sedan",
      "carModel": "Swift Dzire",
      "capacity": 4,
      "price": 1850.00,
      "pricePerKm": 12.50,
      "estimatedDurationMinutes": 180,
      "estimatedDistanceKm": 148,
      "acAvailable": true,
      "driverIncluded": true,
      "cancellationPolicy": "Free cancellation up to 1 hour before pickup"
    }
  ],
  "meta": { "page": 1, "pageSize": 20, "total": 8 }
}
```

Trip types: OneWay · RoundTrip · Outstation · Local

Bookings

Method	Endpoint	Auth	Description
GET	/bookings	Required	User booking history
GET	/bookings/:id	Required	Booking details
POST	/bookings/:id/cancel	Required	Cancel booking
GET	/bookings/:id/invoice	Required	Download invoice

GET /bookings/:id

Returns booking details. The BookingDto includes transport-specific fields for bus bookings:

Field	Type	Description
transportSeatNumbers	string?	Assigned seat number(s)

Field	Type	Description
transportBusNumber	string?	Bus registration number
transportDriverPhone	string?	Driver contact number
transportBoardingPoint	string?	Pickup stop name
transportDroppingPoint	string?	Drop-off stop name

GET /bookings/:id/invoice

Returns a downloadable PDF e-ticket for the authenticated user's booking. For bus bookings, the PDF includes seat number, boarding point, and dropping point.

Response:

- 200 OK
- Content-Type: application/pdf
- Content-Disposition: attachment; filename="<booking-ref>-e-ticket.pdf"

Payments (Razorpay)

Method	Endpoint	Auth	Description
POST	/payments/initiate	Required	Create Razorpay order for a booking
POST	/payments/verify	Required	Verify HMAC signature, confirm booking
GET	/payments/:id	Required	Payment status from Razorpay

POST /payments/initiate

```
Request: { "bookingId": "uuid" }
```

Response 200:

```
{
  "success": true,
  "data": { "orderId": "order_xxx", "amount": 449900, "currency": "INR", "keyId": "rzp_test_xxx" }
}
```

amount is returned in **paise** (₹1 = 100 paise) as required by the Razorpay Checkout SDK.

Falls back to a mock order (with keyId: "") when Razorpay.Enabled = false in config.

POST /payments/verify

```
Request:
{
  "bookingId": "uuid",
  "razorpayOrderId": "order_xxx",
  "razorpayPaymentId": "pay_xxx",
  "razorpaySignature": "hmac_sha256_hex"
}
```

Response 200:

```
{
  "success": true,
}
```

```
"message": "Payment verified successfully",
"data": { "paymentId": "uuid", "bookingRef": "TP20250001" }
}
```

Signature verification: HMAC-SHA256(keySecret, "{orderId}|{paymentId}") — validated server-side via Razorpay's algorithm.

Frontend flow: PaymentPage.tsx loads the Razorpay Checkout SDK (checkout.js via index.html). On pay click it calls /initiate, opens the Razorpay hosted modal pre-set to the chosen method (card / UPI / net banking), then calls /verify with the signature returned by Razorpay on success.

Users (Profile)

Method	Endpoint	Auth	Description
GET	/users/profile	Required	Get profile
PUT	/users/profile	Required	Update profile
GET	/users/travellers	Required	Saved travellers
POST	/users/travellers	Required	Add traveller
DELETE	/users/travellers/:id	Required	Remove traveller
GET	/users/wallet	Required	Wallet balance
POST	/users/wallet/topup	Required	Top up wallet balance
GET	/users/wallet/transactions	Required	Paginated transaction history

POST /users/wallet/topup

```
Request:
{ "amount": 5000.00, "description": "Manual top-up" }

Response 200:
{
  "success": true,
  "data": {
    "walletId": "uuid",
    "balance": 5000.00,
    "currency": "INR"
  }
}
```

Rules: Amount must be > 0 and ≤ ₹1,00,000 per transaction.

GET /users/wallet/transactions

```
Query Params: page=1&pageSize=20

Response 200:
{
  "success": true,
  "data": {
    "items": [
      {
        "id": "uuid",
        "type": "Debit",
        "amount": 4499.00,
        "description": "Flight booking TP2024001234",
        "referenceId": "uuid",
        "createdAt": "2024-12-01T06:30:00Z"
      }
    ]
  }
}
```



```

}
],
"total": 5,
"page": 1,
"pageSize": 20
}
}

```

Wallet Payment in Bookings

Both `/flights/book` and `/hotels/book` accept an optional `useWallet` field:

```

{
  "flightId": "uuid",
  "class": "Economy",
  "passengers": [{ "name": "John Doe", "age": 30, "gender": "Male", "passportNo": "A1234567" }],
  "couponCode": "SAVE100",
  "useWallet": true
}

```

When `useWallet: true`, the `finalAmount` (after coupon discount) is deducted from the user's wallet atomically with the booking. If the wallet balance is insufficient, the API returns HTTP **422** with a descriptive error.

Admin

All admin endpoints require `[Authorize(Roles = "Admin")]`. Enums serialize as strings (e.g. "Confirmed", "Flight") via `JsonStringEnumConverter`.

Method	Endpoint	Auth	Role	Description
GET	<code>/admin/dashboard</code>	Required	Admin	Aggregate stats (users, revenue, bookings)
GET	<code>/admin/analytics</code>	Required	Admin	Monthly revenue + by-type + by-status
GET	<code>/admin/users</code>	Required	Admin	Paginated user list with search
POST	<code>/admin/users/:id/block</code>	Required	Admin	Toggle user active/blocked
GET	<code>/admin/users/:id/overview</code>	Required	Admin	User wallet, status + last 20 bookings
GET	<code>/admin/bookings</code>	Required	Admin	Paginated all bookings with filters
GET	<code>/admin/bookings/export-csv</code>	Required	Admin	Download all bookings as CSV file
GET	<code>/admin/coupons</code>	Required	Admin	All coupons
POST	<code>/admin/coupons</code>	Required	Admin	Create coupon
PUT	<code>/admin/coupons/:id</code>	Required	Admin	Update coupon
DELETE	<code>/admin/coupons/:id</code>	Required	Admin	Deactivate coupon
GET	<code>/admin/coupons/analytics</code>	Required	Admin	Per-coupon usage stats (uses, discount, revenue)
GET	<code>/admin/hotels</code>	Required	Admin	List all registered hotels

Method	Endpoint	Auth	Role	Description
POST	/admin/hotels	Required	Admin	Register hotel + create manager account + send credentials email
POST	/admin/hotels/:id/toggle	Required	Admin	Toggle hotel active/inactive
DELETE	/admin/hotels/reviews/:id	Required	Admin	Hard-delete a hotel review
GET	/admin/flights	Required	Admin	List all seeded flights (searchable)
PUT	/admin/flights/:id	Required	Admin	Edit flight price/seats/times/status

GET /admin/dashboard

```

Response 200:
{
  "success": true,
  "data": {
    "totalUsers": 4,
    "totalBookings": 120,
    "totalRevenue": 548000.00,
    "activeBookings": 95,
    "cancelledBookings": 25,
    "flightBookings": 80,
    "hotelBookings": 40,
    "avgBookingValue": 4566.67
  }
}

```

GET /admin/analytics

```

Response 200:
{
  "success": true,
  "data": {
    "monthlyRevenue": [
      { "month": "Dec 2025", "revenue": 82000.00, "bookingCount": 18 }
    ],
    "bookingsByStatus": [
      { "status": "Confirmed", "count": 95 },
      { "status": "Cancelled", "count": 25 }
    ],
    "bookingsByType": [
      { "type": "Flight", "count": 80, "revenue": 360000.00 },
      { "type": "Hotel", "count": 40, "revenue": 188000.00 }
    ]
  }
}

```

GET /admin/users

Query Params: page=1&pageSize=20&search=john

```

Response 200 (paginated):
{
  "success": true,
  "data": [
    {
      "id": "uuid",
      "name": "John Doe",
      "email": "user@example.invalid",
      "phone": "9876543210",
      "role": "User",
      "isActive": true,

```

```
"isVerified": true,
"walletBalance": 1500.00,
"totalBookings": 5,
"createdAt": "2026-01-01T00:00:00Z"
},
],
"meta": { "page": 1, "pageSize": 20, "total": 4, "totalPages": 1 }
}
```

POST /admin/users/:id/block

Toggles isActive — blocks active users, unblocks blocked users.

```
Response 200:
{ "success": true, "message": "User blocked.", "data": { ...AdminUserDto } }
```

GET /admin/bookings

Query Params: page=1&pageSize=15&status=Confirmed&type=Flight

POST /admin/coupons

```
Request:
{
  "code": "SUMMER25",
  "type": "Percentage",
  "value": 25,
  "minAmount": 3000,
  "maxDiscount": 500,
  "usageLimit": 100,
  "expiresAt": "2026-12-31"
}
```

```
Response 201:
{ "success": true, "message": "Coupon created.", "data": { ...CouponDto } }
```

PUT /admin/coupons/:id

```
Request:
{
  "type": "Percentage",
  "value": 20,
  "minAmount": 2000,
  "maxDiscount": 400,
  "usageLimit": 200,
  "expiresAt": "2026-12-31",
  "isActive": true
}
```

DELETE /admin/coupons/:id

Deactivates the coupon (sets isActive = false). Does not hard-delete.

```
Response 200:
{ "success": true, "message": "Coupon deactivated." }
```

Standard API Response Wrapper

```
{
  "success": true | false,
```

```
"message": "Human-readable message",
"data": { ... } | null,
"errors": [ ] | null,
"meta": {
  "page": 1,
  "limit": 10,
  "total": 100,
  "totalPages": 10
}
```

Hotel Manager

All hotel-manager endpoints require [Authorize(Roles = "Hotel")]. The JWT token for hotel managers includes a hotelId claim automatically scoped to their property.

Method	Endpoint	Auth	Role	Description
GET	/hotel-manager/dashboard	Required	Hotel	Bookings, revenue, rooms overview
GET	/hotel-manager/bookings	Required	Hotel	Paginated bookings for this hotel
GET	/hotel-manager/profile	Required	Hotel	Hotel details + all rooms
PUT	/hotel-manager/profile	Required	Hotel	Update hotel name, description, amenities, images
POST	/hotel-manager/rooms	Required	Hotel	Add a new room type
PUT	/hotel-manager/rooms/:id	Required	Hotel	Update room price, amenities, images
DELETE	/hotel-manager/rooms/:id	Required	Hotel	Soft-delete (deactivate) a room

GET /admin/hotels — List Hotels

```
Response 200:
{
  "success": true,
  "data": [
    {
      "id": "uuid",
      "name": "The Grand Palace",
      "city": "Mumbai",
      "address": "123 Marine Drive",
      "starRating": 5.0,
      "reviewScore": 4.3,
      "reviewCount": 120,
      "isActive": true,
      "roomCount": 6,
      "managerEmail": "manager@example.invalid",
      "createdAt": "2026-01-01T00:00:00Z"
    }
  ]
}
```

POST /admin/hotels/:id/toggle — Toggle Hotel Active

```
Response 200:
{ "success": true, "data": { "id": "uuid", "name": "The Grand Palace", "isActive": false } }
```

DELETE /admin/hotels/reviews/:id — Delete Hotel Review

Permanently removes a hotel review and recalculates the hotel's ReviewScore and ReviewCount.

```
Response 200:
{ "success": true, "message": "Hotel review deleted." }
```

GET /admin/users/:id/overview — View-as-User

Returns a read-only snapshot of a user's account for admin inspection.

```
Response 200:
{
  "success": true,
  "data": {
    "id": "uuid",
    "name": "John Doe",
    "email": "user@example.invalid",
    "phone": "9876543210",
    "role": "User",
    "isActive": true,
    "isVerified": true,
    "walletBalance": 2500.00,
    "createdAt": "2026-01-15T00:00:00Z",
    "recentBookings": [
      {
        "id": "uuid",
        "bookingReference": "FL2026XXXXXX",
        "type": "Flight",
        "status": "Confirmed",
        "finalAmount": 4500.00,
        "bookingDate": "2026-05-01T10:00:00Z"
      }
    ]
  }
}
```

GET /admin/bookings/export-csv — Export Bookings as CSV

Downloads all bookings (optionally filtered by status and type) as a text/csv file.

```
Query Params: status=Confirmed&type=Flight (both optional)

Response 200: Content-Type: text/csv
BookingReference,UserName,UserEmail,Type,Status,Amount,BookingDate,...
FL2026XXXXXX,Sample User,user@example.invalid,Flight,Confirmed,4500,2026-05-01,...
```

GET /admin/coupons/analytics — Coupon Usage Analytics

Returns per-coupon stats aggregated from all bookings.

```
Response 200:
{
  "success": true,
  "data": [
    {
      "id": "uuid",
      "code": "FLYSAVER",
      "totalUses": 42,
      "totalDiscount": 12600.00,
      "totalRevenue": 189000.00
    }
  ]
}
```

GET /admin/flights — List All Flights

Query Params: search=IndiGo (optional – filters on FlightNumber/Airline/Source/Destination)

Response 200:

```
{
  "success": true,
  "data": [
    {
      "id": "uuid",
      "flightNumber": "6E-201",
      "airline": "IndiGo",
      "source": "BOM",
      "destination": "DEL",
      "departureTime": "2026-06-01T06:00:00Z",
      "arrivalTime": "2026-06-01T08:15:00Z",
      "duration": 135,
      "totalSeats": 180,
      "availableSeats": 164,
      "economyPrice": 3500.00,
      "businessPrice": null,
      "stops": 0,
      "isActive": true,
      "createdAt": "2026-05-01T..."
    }
  ]
}
```

PUT /admin/flights/:id — Update Flight

All fields are optional — only provided fields are patched.

Request:

```
{
  "economyPrice": 3200,
  "businessPrice": 8500,
  "totalSeats": 180,
  "availableSeats": 150,
  "departureTime": "2026-06-01T07:00:00",
  "arrivalTime": "2026-06-01T09:30:00",
  "isActive": true
}
```

Response 200:

```
{ "success": true, "data": { ...AdminFlightDto } }
```

POST /admin/hotels — Register Hotel

Request:

```
{
  "hotelName": "The Grand Palace",
  "city": "Mumbai",
  "address": "123 Marine Drive, Mumbai",
  "starRating": 5,
  "managerName": "Rajesh Kumar",
  "managerEmail": "manager@example.invalid",
  "managerPassword": "replace-with-strong-temporary-password"
}
```

Response 201:

```
{
  "success": true,
  "message": "Hotel registered and credentials emailed.",
  "data": {
    "id": "uuid",
    "name": "The Grand Palace",
    "city": "Mumbai",
    "starRating": 5.0,
    "isActive": true,
    "roomCount": 0,
    "managerEmail": "manager@example.invalid"
  }
}
```

GET /hotel-manager/dashboard

```
Response 200:
{
  "success": true,
  "data": {
    "totalBookings": 45,
    "activeBookings": 32,
    "cancelledBookings": 13,
    "totalRevenue": 248500.00,
    "totalRooms": 6,
    "activeRooms": 5,
    "avgReviewScore": 4.3,
    "reviewCount": 120
  }
}
```

POST /hotel-manager/rooms — Add Room

```
Request:
{
  "roomType": "Deluxe King",
  "pricePerNight": 4500,
  "maxGuests": 2,
  "totalRooms": 10,
  "amenities": "[\"AC\", \"WiFi\", \"TV\", \"Mini Bar\", \"Sea View\"]",
  "images": "[\"https://example.com/room1.jpg\", \"https://example.com/room2.jpg\"]"
}

Response 201:
{
  "success": true,
  "message": "Room added.",
  "data": { "id": "uuid", "roomType": "Deluxe King", "pricePerNight": 4500, "isActive": true }
}
```

Operator Management (Admin only)

Method	Endpoint	Auth	Description
GET	/admin/flight-operators	Admin	List all flight operators
POST	/admin/flight-operators	Admin	Register new airline
POST	/admin/flight-operators/{id}/toggle	Admin	Activate/deactivate airline
GET	/admin/bus-operators	Admin	List all bus operators
POST	/admin/bus-operators	Admin	Register new bus operator
POST	/admin/bus-operators/{id}/toggle	Admin	Activate/deactivate bus operator
GET	/admin/cab-operators	Admin	List all cab operators
POST	/admin/cab-operators	Admin	Register new cab operator/driver
POST	/admin/cab-operators/{id}/toggle	Admin	Activate/deactivate cab operator

POST /admin/flight-operators

```
Request:
{
  "companyName": "IndiGo",
  "iataCode": "6E",
  "logoUrl": null,
}
```

```
"headquartersCity": "Gurugram",
"contactPhone": "+91-124-4973838",
"managerEmail": "operator@example.invalid",
"managerPassword": "replace-with-strong-temporary-password",
"managerName": "Anil Sharma"
}
```

Response 201:

```
{
  "success": true,
  "message": "Flight operator registered and credentials emailed.",
  "data": {
    "id": "uuid",
    "name": "IndiGo",
    "iataCode": "6E",
    "isActive": true,
    "flightCount": 0,
    "managerEmail": "operator@example.invalid",
    "createdAt": "2026-05-19T..."
  }
}
```

Flight Operator Portal

([Authorize(Roles="FlightOperator")])

Method	Endpoint	Auth	Description
GET	/flight-operator/dashboard	FlightOperator	Stats overview
GET	/flight-operator/flights	FlightOperator	List operator's flights
POST	/flight-operator/flights	FlightOperator	Add a new flight
PUT	/flight-operator/flights/{id}	FlightOperator	Edit a flight
DELETE	/flight-operator/flights/{id}	FlightOperator	Remove a flight
GET	/flight-operator/bookings	FlightOperator	View passenger bookings

POST /flight-operator/flights

```
Request:
{
  "flightNumber": "6E-201",
  "source": "BOM",
  "destination": "DEL",
  "departureTime": "2026-06-01T06:00:00",
  "arrivalTime": "2026-06-01T08:15:00",
  "totalSeats": 180,
  "economyPrice": 3500,
  "businessPrice": null,
  "stops": 0,
  "layoverAirport": null,
  "layoverDurationMinutes": null
}
```

Validation rules:

- source and destination must differ
- arrivalTime must be after departureTime
- stops must be 0 or 1 (only non-stop and 1-stop flights supported)
- When stops = 1: layoverAirport is required and must differ from source/destination; layoverDurationMinutes must be > 0

- When stops = 0: layoverAirport and layoverDurationMinutes are ignored

layoverAirport is stored uppercase (e.g., "HYD").

Bus Operator Portal ([Authorize(Roles="BusOperator")])

Method	Endpoint	Auth	Description
GET	/bus-operator/dashboard	BusOperator	Stats + company overview (buses, revenue, passengers)
GET	/bus-operator/buses	BusOperator	List all operator's buses
POST	/bus-operator/buses	BusOperator	Add a new bus
PUT	/bus-operator/buses/{busId}	BusOperator	Edit bus details
DELETE	/bus-operator/buses/{busId}	BusOperator	Soft-delete (deactivate) a bus
GET	/bus-operator/bookings	BusOperator	View passenger bookings
GET	/bus-operator/buses/by-date?date=	BusOperator	Buses scheduled on a date (respects OneTime/Daily/Weekly)
GET	/bus-operator/buses/{busId}/seat-layout	BusOperator	Full seat map with booked passengers
PUT	/bus-operator/buses/{busId}/seat-layout	BusOperator	Update layout config / ladies seats
POST	/bus-operator/buses/{busId}/bulk-book	BusOperator	Bulk-book specific seats for a passenger
DELETE	/bus-operator/bookings/{bookingId}	BusOperator	Cancel booking + issue 90% wallet refund

POST /bus-operator/buses

```
Request:
{
  "busNumber": "KA01AB1234",
  "origin": "Bangalore",
  "destination": "Mumbai",
  "departureTime": "2026-06-01T21:00:00",
  "arrivalTime": "2026-06-02T09:00:00",
  "totalSeats": 40,
  "price": 800,
  "busType": "AC Seater",
  "seatLayoutConfig": "2-2",
  "seatRows": 10,
  "ladiesSeats": ["1A", "1B"],
  "amenities": ["WiFi", "Charging Point", "Water Bottle"],
  "driverName": "Ravi Kumar",
  "driverPhone": "+91 9876543210",
  "driverLicense": "KA01 20190001234",
  "photoUrl": "https://example.com/bus.jpg",
  "scheduleType": "Weekly",
  "daysOfWeek": ["Monday", "Wednesday", "Friday"],
  "boardingPoints": "Majestic, Electronic City",
  "droppingPoints": "Dadar, Thane"
}

Response 201:
{ "success": true, "message": "Bus added.", "data": { ...OperatorBusDto } }
```

Schedule types: OneTime (single departure date), Daily (runs every day), Weekly (runs on specified days of week).

Seat layouts: 2-2 (standard seater), 2-1 (sleeper/luxury), 1-1 (VIP), 3-2 (economy), 2-2-2 (wide-body).

GET /bus-operator/buses/{busId}/seat-layout

Returns the full seat grid showing which seats are booked (with passenger name, email, phone, booking ref) and which are available, ladies-only, or unassigned.

```
Response 200:
{
  "success": true,
  "data": {
    "busId": "uuid",
    "busNumber": "KA01AB1234",
    "origin": "Bangalore",
    "destination": "Mumbai",
    "departureTime": "2026-06-01T21:00:00Z",
    "layoutConfig": "2-2",
    "seatRows": 10,
    "ladiesSeats": ["1A", "1B"],
    "seats": [
      { "seatNumber": "1A", "isLadiesOnly": true, "isBooked": false },
      { "seatNumber": "3B", "isLadiesOnly": false, "isBooked": true, "passengerName": "Sample Passenger", "passengerEmail": "passenger@example.invalid", "bookingRef": "BU2026000001", "bookingId": "uuid" }
    ],
    "unassignedPassengers": 2
  }
}
```

POST /bus-operator/buses/{busId}/bulk-book

```
Request:
{
  "passengerName": "Priya Sharma",
  "passengerEmail": "passenger@example.invalid",
  "passengerPhone": "+91 9123456789",
  "seatNumbers": ["3A", "3B"]
}

Response 201:
{ "success": true, "message": "Seats booked successfully.", "data": { ...OperatorBookingDto } }
```

Cab Operator Portal ([Authorize(Roles="CabOperator")])

Method	Endpoint	Auth	Description
GET	/cab-operator/dashboard	CabOperator	Stats + driver overview
GET	/cab-operator/bookings	CabOperator	View passenger bookings

Version 2 — Phase 2 Changes (2026-05-20)

Bus Seat Locking Endpoints

POST /buses/:busId/seats/lock — Lock seats while user is selecting (10-min TTL, auto-expired)

```
Request: { "seatNumbers": ["3", "7"] }
Response: 200 { "success": true, "message": "Seats locked." }
```

DELETE /buses/:busId/seats/lock — Release locks when user deselects or navigates away

```
Request: { "seatNumbers": ["3"] }
Response: 200 { "success": true, "message": "Seats unlocked." }
```

GET /buses/:busId/seats/locked — Poll locked seats (excludes current user's own locks)

```
Response: { "success": true, "data": [3, 7, 12] }
```

Seats locked by others shown in amber on the seat map. Booking rejected if a conflict is detected server-side.

Hotel Images Gallery

GET /hotels/:id now returns images (JSON string array of gallery URLs) alongside imageUrl:

```
{
  "imageUrl": "https://.../main.jpg",
  "images": "[\"https://.../1.jpg\", \"https://.../2.jpg\", \"https://.../3.jpg\", \"https://.../4.jpg\", \"https://.../5.jpg\"]"
}
```

images is null for Amadeus external results (falls back to imageUrl).

Error Codes

Code	Description
400	Validation error
401	Unauthorized
403	Forbidden (role)
404	Resource not found
409	Conflict (duplicate)
422	Business rule violation
429	Rate limit exceeded
500	Internal server error

Notifications (Phase 3)

Method	Endpoint	Auth	Description
GET	/api/v1/users/notifications	User	List all notifications (newest first)
GET	/api/v1/users/notifications/unread-count	User	Returns unread count as integer
PATCH	/api/v1/users/notifications/:id/read	User	Mark single notification as read
PATCH	/api/v1/users/notifications/read-all	User	Mark all notifications as read

Notification types: BookingConfirmed, BookingCancelled, CouponExpiring, PriceDrop

Price Alerts (Phase 3)

Method	Endpoint	Auth	Description
GET	/api/v1/users/price-alerts	User	List active price alerts
POST	/api/v1/users/price-alerts	User	Create alert (deduplicates by route+date)
DELETE	/api/v1/users/price-alerts/:id	User	Deactivate alert

POST body: { origin, destination, travelDate, currentPrice, email? }

Featured Coupons (Phase 3)

Method	Endpoint	Auth	Description
GET	/api/v1/coupons/featured	None	Returns active featured non-expired coupons

Announcements (Phase 4)

Site-wide announcement banners created by admins. Active banners are fetched on every page load via the public endpoint and shown as dismissable top banners in the UI.

Method	Endpoint	Auth	Description
GET	/announcements/active	None	List currently active (non-expired) banners
GET	/announcements	Admin	List all announcements (including paused)
POST	/announcements	Admin	Create a new announcement banner
PUT	/announcements/:id	Admin	Update message/type/expiry or pause/resume
DELETE	/announcements/:id	Admin	Soft-delete an announcement

GET /announcements/active

Returns all active, non-expired announcements. No auth required — called on every page load.

```
Response 200:
{
  "success": true,
  "data": [
    {
      "id": "uuid",
      "message": "Monsoon Sale: Flat 20% off all flights this week!",
      "type": "info",
      "expiresAt": "2026-06-01T23:59:59Z",
      "isActive": true,
      "createdAt": "2026-05-21T..."
    }
  ]
}
```

```
}
```

Types: info (blue banner) · warning (yellow banner) · success (green banner)

POST /announcements

```
Request:
{
  "message": "Scheduled maintenance on 25 May between 2-4 AM IST.",
  "type": "warning",
  "expiresAt": "2026-05-25T04:00:00"
}

Response 201:
{ "success": true, "data": { ...AnnouncementDto } }
```

expiresAt is optional — omit for a banner that stays until manually paused or deleted.

PUT /announcements/:id

```
Request (all fields optional):
{
  "message": "Updated message text",
  "type": "success",
  "expiresAt": "2026-06-30T23:59:59",
  "isActive": false
}

Response 200:
{ "success": true, "data": { ...AnnouncementDto } }
```

Set isActive: false to pause without deleting; isActive: true to resume.

DELETE /announcements/:id

```
Response 200:
{ "success": true, "message": "Announcement deleted." }
```

AI Endpoints

All AI endpoints proxy to Groq server-side. Set Groq:ApiKey only in ignored local settings, user secrets, or deployment secrets. All endpoints return graceful fallbacks when the key is missing.

Method	Endpoint	Auth	Description
POST	/ai/chat	None	Streaming SSE chat with travel assistant
POST	/ai/nl-search	None	Parse natural language query -> structured search params
POST	/ai/recommendations	None	Get 4 AI destination recommendations
POST	/ai/trip-plan	None	Streaming SSE trip itinerary generator
GET	/ai/price-insight	None	One-line price trend tip for a flight route (cached 30 min)

POST /ai/chat

Streams the Groq response as text/event-stream. Each event is data: "text chunk"\n\n. Stream ends with data: [DONE]\n\n.

```
Request:
{
  "messages": [
    { "role": "user", "content": "Find me cheap flights to Goa" },
    { "role": "assistant", "content": "I can help! Head to /flights and search..." },
    { "role": "user", "content": "What's the best time to visit?" }
  ]
}

Response: text/event-stream
data: "Goa is best visited between November and February"
data: " when the weather is dry and cool."
data: [DONE]
```

POST /ai/nl-search

Parses a free-text travel query into structured search parameters.

```
Request:
{ "query": "Flights from Mumbai to Delhi tomorrow for 2 passengers" }

Response 200:
{
  "type": "flight",
  "origin": "BOM",
  "originCity": "Mumbai",
  "destination": "DEL",
  "destinationCity": "Delhi",
  "date": "2026-05-23",
  "returnDate": null,
  "passengers": 2,
  "hotelCity": null,
  "checkIn": null,
  "checkOut": null,
  "guests": null
}
```

Supported types: flight · hotel · bus · train · cab

POST /ai/recommendations

Returns 4 AI-curated destination recommendations. Optionally pass booking history to personalise suggestions.

```
Request:
{ "bookingHistory": ["Mumbai", "Delhi"] }

Response 200:
[
  {
    "city": "Goa",
    "tagline": "Sun, sand & vibes",
    "reason": "A perfect beach escape complementing your city trips.",
    "bestFor": "Beaches",
    "flightCode": "GOI"
  },
  ...
]
```

POST /ai/trip-plan

Streams a Groq-generated day-by-day itinerary as text/event-stream. Response is markdown text with embedded [BOOK_FLIGHT:params] and [BOOK_HOTEL:params] deep-link markers.

```
Request:
{ "brief": "5-day Goa beach trip for a couple, budget ₹30,000" }

Response: text/event-stream
data: "## Trip Overview\n"
data: "A romantic 5-day Goa escape..."
data: "\n## Day 1: Arrival\n"
...
data: "- **Flight**: Mumbai to Goa -> [BOOK_FLIGHT:origin=BOM&destination=GOI&date=2026-06-01]\n"
data: [DONE]
```

GET /ai/price-insight

Returns a one-sentence price trend tip for a specific flight route. Cached server-side for 30 minutes.

```
GET /ai/price-insight?origin=BOM&destination=DEL
```

```
Response 200:
{ "insight": "Prices are lowest on Tuesday and Wednesday on this route." }
```

Returns { "insight": null } when the AI key is not configured.